

Contents

guide 01_camel	1
0. 这篇论文一句话	2
1. 论文出现时的语境	2
2. 论文结构地图	3
3. 论文想解决的真实痛点	4
4. 核心概念	5
4.1 Communicative agents	5
4.2 Role-playing	5
4.3 Task specifier	5
4.4 AI user	5
4.5 AI assistant	6
4.6 Inception prompting	6
4.7 Critic-in-the-loop	6
5. Figure 1 的系统流程	6
6. 论文中的形式化动态	7
7. Figure 2 的 inception prompt 设计	7
8. CAMEL 的四类失败模式	8
8.1 Role flipping	8
8.2 Assistant repeats instruction	8
8.3 Flake replies	9
8.4 Infinite loop	9
9. Termination conditions	9
10. 成本为什么接近二次增长	9
11. 数据生成: AI Society 和 Code	10
12. Table 1: CAMEL agent solution vs single-shot	10
13. Table 2: 逐步微调和 knowledge emergence	11
14. Table 3: HumanEval 和 HumanEval+	12
15. Critic-in-the-loop	12
16. 方法总图	12
17. 本地代码映射	13
18. 最小代码片段	14
19. 30 到 60 分钟本地实验	15
20. 证据链怎么读	15
21. 局限性	16
22. 对现代 multi-agent 框架的意义	17
23. 和第 25 篇的连接	18
24. 用 AI agent 正确学习这篇	18
25. 闭卷掌握检查	18
26. 最小复述模板	19

guide_01_camel

原论文: CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society

本地原文 PDF: [learning/multi-agent-orchestration/paper/01_camel.pdf](#)

作者: Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin,

Bernard Ghanem

机构: King Abdullah University of Science and Technology

会议版本: NeurIPS 2023

arXiv: 2303.17760

类型: multi-agent systems, role-playing, communicative agents, synthetic data generation

0. 这篇论文一句话

CAMEL 把“一个人不断提示 ChatGPT 完成复杂任务”改造成“两个带角色的 LLM agent 自动对话完成任务”，并用 inception prompting、task specifier、assistant/user role assignment、termination rules 和 critic-in-the-loop 来约束这种自动协作。

它的重要性不是证明多 agent 永远更好，而是把一个新研究对象摆到桌面上：

LLM agent 和 LLM agent 之间的自然语言协作轨迹，
能否被稳定生成、被分析、被评测、被用作训练数据？

你读这篇要同时抓住两面。

正面价值：

- 角色让 agent 有分工。
- 对话让任务被逐步分解。
- task specifier 把模糊想法变成具体任务。
- 自动对话可以大规模生成 instruction-solution 数据。
- 多 agent 协作能产生比 single-shot 更细的方案。

反面警告：

- 角色会翻转。
- assistant 会重复 instruction。
- agent 会给空泛承诺。
- 对话会陷入循环。
- 成本随对话长度快速增长。
- GPT4/human evaluation 可能偏好长答案。
- Misalignment 数据说明 autonomous agent 可以放大有害意图。

如果第 25 篇 AI Agents That Matter 教你“不要轻信 agent 分数”，CAMEL 则教你“多 agent 协作到底是怎么被构造出来的”。

1. 论文出现时的语境

2023 年，chat-based LLM 已经能做很多复杂任务。用户可以通过多轮对话让模型写代码、规划项目、解释概念、生成方案。问题是：这种成功高度依赖人类不断提供好提示。

一个真实任务往往不是一句 prompt 就能解决。

例如“开发一个 stock trading bot”。普通用户可能不知道应该先让模型安装哪些库、怎么接 Twitter API、怎么做 sentiment analysis、怎么接 market data、怎么设计交易规则、怎么测试风险。模型也需要用户不断把任务往正确方向推。

这就产生 CAMEL 的核心问题：

能不能让另一个 AI agent 代替人类，持续给 assistant 下达合理指令？

如果可以，那么一个模糊想法就能变成一段自动协作轨迹：

```
human idea
-> task specifier
-> AI user gives instruction
-> AI assistant gives solution
-> AI user gives next instruction
-> ...
-> task completed
```

这和 ReAct 的区别在于：ReAct 是单 agent 在 thought/action/observation 间循环。CAMEL 是两个角色化 agent 用自然语言互相驱动，一个像 planner/user，一个像 executor/assistant。

这和 Self-Instruct 的区别在于：Self-Instruct 主要用模型生成 instruction 数据。CAMEL 生成的是 multi-turn instruction-following conversation，可以保存每一轮 instruction-solution pair。

这和后来的 AutoGen、CrewAI、MetaGPT、Magentic-One 的关系是：CAMEL 是早期把角色、对话协议和 multi-agent collaboration 作为核心对象研究的代表之一。

2. 论文结构地图

第 1 节 Introduction:

- 说明 chat LLM 完成复杂任务仍依赖人类引导。
- 提出 autonomous communicative agents 的问题。
- 列出初步观察到的失败模式：role flipping、assistant repeating instructions、flake replies、infinite loop。
- 介绍 role-playing 和 inception prompting。
- 说明会生成 AI Society、Code、Math、Science、Misalignment 等数据。

第 2 节 Related Work:

- 连接 communicative agents、cooperative AI、prompt engineering、instruction tuning、AI alignment。
- 把 CAMEL 放在“自然语言通信的多 agent 系统”和“自动生成 instruction data”之间。

第 3 节 Methodology:

- 第 3.1 节 Role-playing Framework。
- Figure 1 展示 human idea、task specifier、AI user、AI assistant 的流程。
- 定义 AI user 生成 instruction，AI assistant 生成 solution 的对话动态。
- 提出 Critic-In-The-Loop 扩展。
- 第 3.2 节 Inception Prompting。
- Figure 2 展示 AI Society 的 task specifier prompt、assistant system prompt、user system prompt。

第 4 节 Experiments:

- 说明使用两个 gpt-3.5-turbo agents 做 assistant-user cooperation。
- 生成 AI Society 和 Code conversational datasets。
- 生成 Math 和 Science 单轮 QA 数据。
- 分析 role-playing 的四个挑战。
- 说明 termination conditions。

第 5 节 Evaluation:

- 第 5.1 节 Agent Evaluation。
- Table 1 比较 CAMEL agent solution 和 gpt-3.5-turbo single-shot solution。
- 第 5.2 节用 GPT4 评估 fine-tuned LLaMA 7B 的知识 emergence。
- Table 2 展示逐步加入 AI Society、Code、Math、Science 数据后的结果。
- 第 5.3 节 HumanEval 和 HumanEval+。
- Table 3 比较 CAMEL-7B、LLaMA-7B、Vicuna-7B、gpt-3.5-turbo。

第 6 节 Conclusion:

- 总结 role-playing 框架。
- 强调 autonomous cooperation 的挑战: conversation deviation、role flipping、termination conditions。
- 强调开源库和数据生成流程。

附录:

- 展示完整对话样例。
- 展示 misalignment scenario。
- 展示 Code、Math、Science、Embodied Agent 等更多 prompt 和样例。
- 展示 Figure 8/9 的 termination reason 和 prompt ablation。
- 展示 critic-in-the-loop 的 prompt 和 tree-search-like 过程。
- 展示风险、局限、license、human subjects 等 checklist。

3. 论文想解决的真实痛点

单次 prompt 的瓶颈:

- 用户必须知道下一步问什么。
- 用户必须判断模型回答是否足够具体。
- 用户必须不断纠偏。
- 用户必须有领域知识。

单 agent 自动规划的瓶颈:

- 一个 agent 既要规划又要执行，容易混乱。
- 它可能在自己的回答里假装已经完成动作。
- 它缺少一个外部角色持续提出下一步要求。

CAMEL 的设计直觉是:

把人类引导者的角色变成 AI user。

把执行方案的角色保留给 AI assistant。

让二者在明确协议下对话。

这相当于把复杂任务拆成两个职责:

- AI user: planner, instructor, task manager。
- AI assistant: executor, solver, implementer。

如果这个机制稳定，就能用低人力成本收集大量多轮协作数据。

4. 核心概念

4.1 Communicative agents

Communicative agent 是能通过语言和其他 agent 交流的 agent。这里的通信不是内部 hidden state，而是自然语言 message。

CAMEL 关心的是 cooperative setting，也就是两个 agent 有共同目标。它不主要研究对抗博弈，而是研究怎样让两个 LLM 通过对话完成一个任务。

4.2 Role-playing

Role-playing 是给 agent 分配社会角色和任务角色。

例如：

- AI assistant: Python Programmer.
- AI user: Stock Trader.
- idea: develop a trading bot.

这个角色分配会影响 agent 的行为边界。AI user 更像需求方和规划者，AI assistant 更像执行者和方案生成者。

4.3 Task specifier

Task specifier 是一个把模糊 idea 具体化的模块。

输入：

idea:

develop a trading bot for the stock market

roles:

Python Programmer

Stock Trader

输出：

specified task:

develop a trading bot with sentiment analysis,
monitor social media for positive or negative comments,
execute trades based on sentiment analysis results

这个模块的动机是：非专家用户可能只会给模糊目标，但 agent 对话需要具体任务才能稳定推进。

4.4 AI user

AI user 不是真人用户，而是扮演 user role 的 LLM agent。它负责持续给 assistant instruction。

它的职责：

- 一次只给一个 instruction。
- 必要时提供 input。
- 根据 assistant 的回答决定下一步。
- 认为任务完成时输出 CAMEL_TASK_DONE。

4.5 AI assistant

AI assistant 是扮演 assistant role 的 LLM agent。它负责给出具体 solution。

它的职责:

- 不要反过来指挥 user。
- 不要要求 user 做 assistant 的工作。
- 对每个 instruction 给出具体解决方案。
- 不能完成时要诚实拒绝并解释原因。
- 回答结束时请求下一步。

4.6 Inception prompting

Inception prompting 是论文对初始 prompt 工程的命名。它不是每轮都人工写 prompt，而是在对话开始前把三个 prompt 配好:

- task specifier prompt PT。
- assistant system prompt PA。
- user system prompt PU。

一旦对话开始，两个 agent 会自动互相 prompt，直到 termination。

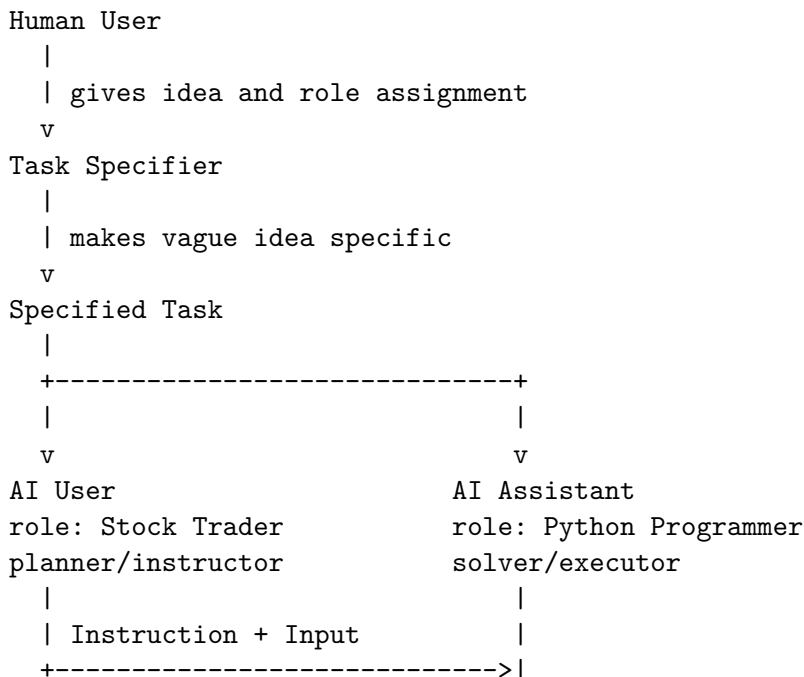
4.7 Critic-in-the-loop

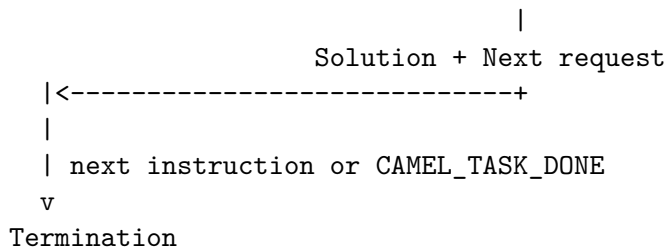
Critic 是可选的第三方角色。它可以是 AI，也可以是人。它从多个 proposal 中选择一个，或者给反馈。

论文说这使得 role-playing 有点像 tree-search-like decision-making。关键不是严格复现 MCTS，而是把“选择哪条对话分支继续”交给 critic。

5. Figure 1 的系统流程

Figure 1 是整篇论文最重要的结构图。可以重建成:





这个图背后的设计理由很朴素:

- 模糊 idea 先具体化。
- 具体任务同时进入两个 agent 的 system prompt。
- AI user 负责“下一步做什么”。
- AI assistant 负责“这一步怎么做”。
- 对话记录成为可分析的数据。

6. 论文中的形式化动态

论文把一轮对话表示为 instruction 和 solution 的 pair。

用导读符号写:

```

I_t = user instruction at turn t
S_t = assistant solution at turn t
M_t = [(I_0, S_0), ..., (I_t, S_t)]

```

下一轮:

```

I_{t+1} = U(M_t)
S_{t+1} = A(M_t, I_{t+1})
M_{t+1} = M_t + [(I_{t+1}, S_{t+1})]

```

其中:

- U 是 AI user agent。
- A 是 AI assistant agent。
- M_t 是到当前为止的 conversation history。

这个公式很简单, 但非常关键。它告诉你 CAMEL 的“状态”不是一个隐式黑盒, 而是不断增长的 message history。

这带来两个工程后果。

第一, 成本会增长。每一轮通常要把历史塞进上下文, 所以对话越长, input tokens 越多。

第二, 错误会积累。早期 instruction 模糊、assistant 答偏、角色翻转, 都可能进入后续 history, 影响后续所有 turn。

7. Figure 2 的 inception prompt 设计

Figure 2 展示 AI Society role-playing 的三个 prompt。

Task specifier prompt:

```

Here is a task that ASSISTANT_ROLE will help USER_ROLE complete:
TASK.
Please make it more specific.

```

Be creative and imaginative.

Reply only with the specified task.

Assistant system prompt 的关键块:

- Never forget you are ASSISTANT_ROLE and I am USER_ROLE.
- Never flip roles.
- Never instruct me.
- If you cannot perform an instruction due to physical, moral, legal, or capability reasons, decline honestly.
- Unless I say the task is completed, always start with Solution:.
- The solution should be specific and include implementations and examples.
- Always end with Next request..

User system prompt 的关键块:

- Never forget you are USER_ROLE and I am ASSISTANT_ROLE.
- Give one instruction at a time.
- Use an Instruction: and optional Input: format.
- Do not ask questions as the main behavior.
- Keep giving instructions until the task is done.
- When done, reply only with CAMEL_TASK_DONE.

这些 prompt 不是装饰, 而是为了解决实际失败。

- " Never flip roles" 对抗 role flipping.
- " Never instruct me" 防止 assistant 抢 user 角色.
- " Solution:" 约束输出具体执行结果.
- " Next request." 让 assistant 把控制权交回 user.
- CAMEL_TASK_DONE 让对话有明确终点.
- moral/legal/capability refusal 试图降低 harmful output.

8. CAMEL 的四类失败模式

论文不是只展示漂亮样例。它很早就承认 autonomous cooperation 会出问题。

8.1 Role flipping

Role flipping 是 assistant 和 user 交换角色。

典型症状:

- assistant 开始给 user 下 instruction.
- user 开始顺从 assistant.
- 原本的 planner/executor 边界消失.

为什么会发生? 因为 LLM 对话模型学过大量人类对话文本, 角色边界不天然稳定。如果 assistant 的问答以问题或指令收尾, 就可能把控制权拿走。

8.2 Assistant repeats instruction

assistant 没有解决问题, 只是复述 user 的 instruction。

这看起来像听懂了, 但没有实际推进任务。它会产生空转数据。

8.3 Flake replies

flake reply 常常是“ I will ...” 形式。assistant 承诺会做某事，但没有真的给 solution。

例如：

I will analyze the data and provide a report.

这不等于完成了分析。CAMEL 用“ Solution:” 和“ specific implementations and examples” 来压制这种空话。

8.4 Infinite loop

两个 agent 会陷入无意义循环，比如互相感谢、说再见、确认已经卡住但无法退出。

这类失败对成本很危险，因为如果没有 max message 或 token limit，对话可能持续消耗 API 调用。

9. Termination conditions

论文设置多种终止条件。

User no instruct:

- 如果 user 连续若干轮没有给 instruction，就结束。

Assistant instruct:

- 如果 assistant 开始 instruct user，说明角色翻转，结束。

End of task token:

- user 输出 CAMEL_TASK_DONE，结束。

Assistant or user token limit:

- 任一 agent 达到上下文/token 限制，结束。

Maximum number of messages:

- 最大 40 messages。
- 这个限制和成本直接相关。

论文附录里还指出，如果终止失败，对话可能一直说 thank you 或 welcome，直到 token limit，可能造成大量 API 调用和很高成本。

这里和第 25 篇形成强连接：

multi-agent capability 必须和 cost guard 一起设计。

10. 成本为什么接近二次增长

每一轮 agent 调用通常都看见历史。假设每轮新增 message 长度大致为 m ，第 t 轮输入长度约为 $t * m$ 。

总输入 tokens 近似：

$$\begin{aligned} \text{input_total} &= \\ &m * (1 + 2 + 3 + \dots + T) \\ &= m * T * (T + 1) / 2 \end{aligned}$$

所以轮数增加时，成本不是线性增长那么简单。输出 tokens 也随轮数增长。

这就是为什么 CAMEL 设置最大消息数，也为什么本仓库已有 `cost_analyzer.py`，并新增 `camel_role_play.py` 用 `max_messages` 模拟保护。

11. 数据生成: AI Society 和 Code

CAMEL 不只是一个对话 demo。论文把 role-playing 当作 synthetic data generation 方法。

AI Society dataset 生成流程:

```
generate assistant roles
generate user roles
generate tasks for role combinations
task specifier makes tasks concrete
AI user and AI assistant role-play
save conversation trajectory
```

论文报告 AI Society 中:

- 50 assistant roles。
- 50 user roles。
- 每个 role combination 生成 10 tasks。
- 总计 25,000 conversations。

Code dataset 类似，但 prompt 针对编程语言和代码任务做了额外工程。

Math 和 Science 数据不同，它们是 single-turn question-answer datasets，用于研究 fine-tuned model 的知识 emergence。

Misalignment dataset 用来模拟潜在有害应用，展示未对齐 autonomous agent system 的风险。导读不复述其可操作细节，因为重点是安全结论: 多 agent 自动协作可以放大不良目标，必须有拒绝、审核、终止和人类监督。

12. Table 1: CAMEL agent solution vs single-shot

Table 1 比较 CAMEL agent solution 和 gpt-3.5-turbo single-shot solution。

评估流程:

- 从 AI Society 随机抽 100 tasks。
- 从 Code 随机抽 100 tasks。
- 用 GPT4 总结 CAMEL 多轮对话得到 final solution。
- 让评估者比较 CAMEL summarized solution 和 single-shot solution。
- AI Society 做 human evaluation。
- AI Society 和 Code 做 GPT4 evaluation。

结果:

AI Society human evaluation:

- Draw: 13.3%。
- gpt-3.5-turbo wins: 10.4%。
- CAMEL agents win: 76.3%。

AI Society GPT4 evaluation:

- Draw: 4.0%。
- gpt-3.5-turbo wins: 23.0%。
- CAMEL agents win: 73.0%。

Code GPT4 evaluation:

- Draw: 0.0%。
- gpt-3.5-turbo wins: 24.0%。
- CAMEL agents win: 76.0%。

这个结果支持的结论:

- 多轮 role-playing solution 往往比 single-shot 更细。
- 在这些 sample tasks 上, human 和 GPT4 evaluation 都偏向 CAMEL。
- 对复杂任务, 逐步 instruction-solution 可能产生更完整方案。

这个结果没有证明的结论:

- 没有证明 CAMEL 在所有真实任务上更好。
- 没有证明成本受控后仍一定更优。
- 没有证明 GPT4 judge 没有偏好长答案。
- 没有证明自动对话不会引入 hallucination。
- 没有证明多 agent 架构比强 single-agent multi-turn baseline 更优。

从第 25 篇的视角看, CAMEL 的证据很有价值, 但还需要 cost-controlled baseline 和更严格 task completion evaluation。

13. Table 2: 逐步微调和 knowledge emergence

论文还用生成数据微调 LLaMA 7B。

数据加入顺序大致是:

AI Society

AI Society + Code

AI Society + Code + Math

AI Society + Code + Math + Science

评估任务:

- 20 AI Society tasks。
- 20 coding tasks。
- 20 math tasks。
- 60 science tasks。

Table 2 的主张是: 当模型逐步加入不同领域数据后, 在对应领域上的回答质量上升。论文用 GPT4 agent 比较不同 fine-tuned model 的回答质量。

值得注意的解释:

- Code 数据可能也改善 Science, 因为 Code 数据里有科学相关任务。
- AI Society 可能改善 Code, 因为 AI Society 包含 programmer role。
- 有些 draw 是“两个答案都差”, 有些 draw 是“两个答案都好”, 不能只看 draw 数字。

对新手来说, 这节要和数据混合学习联系起来: synthetic conversations 不只是给 agent demo 看, 还可能作为 instruction-tuning data。

14. Table 3: HumanEval 和 HumanEval+

论文用 HumanEval 和 HumanEval+ 测 CAMEL-7B 的 coding 能力。

结果简化记:

- gpt-3.5-turbo 仍远强于 7B 模型。
- LLaMA-7B 在 HumanEval pass@1 约 10.5。
- Vicuna-7B 在 HumanEval pass@1 约 11.0。
- CAMEL-7B 在 HumanEval pass@1 约 14.0。
- CAMEL-7B 在 HumanEval pass@100 约 57.9。
- HumanEval+ 上 CAMEL-7B 也优于 Vicuna-7B。

这说明 CAMEL 数据对 7B code capability 有帮助，但不要误读成它接近 gpt-3.5-turbo。

这里最重要的是证据边界:

CAMEL data improves a small fine-tuned model relative to similar 7B baselines.
It does not replace frontier models.

15. Critic-in-the-loop

Critic-in-the-loop 是 CAMEL 的重要扩展。

基本想法:

```
user agent proposes options
assistant agent proposes options
critic agent or human selects one
selected branch continues
```

这类似 tree search:

- expansion: 生成多个候选。
- selection: critic 根据 criteria 选择。
- continuation: 沿选择分支继续。

但它不是严格数学化的 MCTS。critic 的 selection criteria 来自 prompt 或人类偏好。

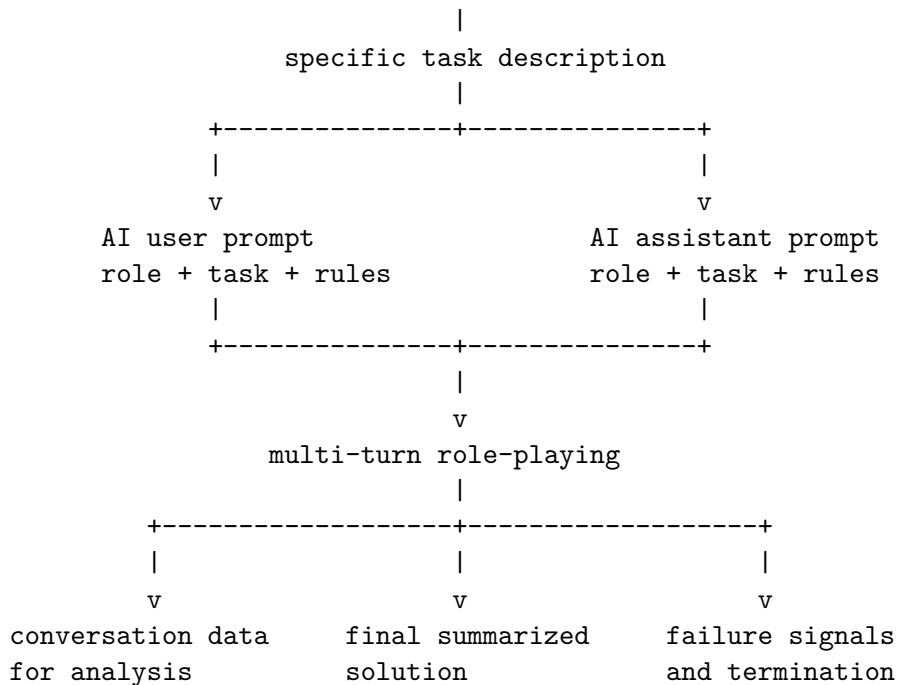
这个设计后来在很多 agent 框架里都能看到影子:

- reviewer agent.
- judge agent.
- supervisor.
- debate judge.
- human approval.

本仓库的 conflict_resolution.py、debate.py、hierarchical.py 和 capstone_coding_crew.py 都是这类思想的教学版本。

16. 方法总图

```
human gives only a rough idea
|
v
task specifier agent
```



更细的数据流:

PT: task specifier prompt
 PA: assistant system prompt
 PU: user system prompt

idea, roles
 -> PT
 -> specified_task
 -> PA and PU
 -> M_0

for each turn t:
 I_t = U(M_{t-1})
 S_t = A(M_{t-1}, I_t)
 M_t = M_{t-1} + [(I_t, S_t)]

stop if:
 user says CAMEL_TASK_DONE
 assistant starts instructing user
 token limit reached
 max message limit reached

17. 本地代码映射

新增文件:

learning/multi-agent-orchestration/src/camel_role_play.py

它对应论文机制:

- specify_task: task specifier.

- inception_prompts: PT、PA、PU 的简化版本。
- plan_instructions: AI user 的 instruction sequence。
- assistant_solution: AI assistant 的 solution format。
- detect_failures: role flipping、assistant repeats instruction、flake reply、infinite loop。
- run_role_play: 多轮 role-playing loop。
- max_messages: 成本保护和 termination condition。

已有本地文件:

```
learning/multi-agent-orchestration/src/debate.py
learning/multi-agent-orchestration/src/hierarchical.py
learning/multi-agent-orchestration/src/message_bus.py
learning/multi-agent-orchestration/src/conflict_resolution.py
learning/multi-agent-orchestration/src/cost_analyzer.py
learning/multi-agent-orchestration/src/capstone_coding_crew.py
```

它们分别对应:

- debate: 多 agent 互相 critique。
- hierarchical: supervisor + workers。
- message_bus: 多 agent 消息传递。
- conflict_resolution: vote、weighted vote、judge、borda。
- cost_analyzer: 多 agent 成本膨胀。
- capstone_coding_crew: PM、Engineer、Reviewer 的三角色协作。

18. 最小代码片段

```
from camel_role_play import run_role_play

result = run_role_play(
    idea="build a paper-reading agent",
    assistant_role="Python engineer",
    user_role="LLM researcher",
)

print(result.spec.specified_task)
print(result.termination_reason)
print(result.failures)
for turn in result.turns:
    print(turn.instruction)
    print(turn.solution)
```

你应该观察到:

- 模糊 idea 被 task specifier 具体化。
- AI user 逐步给出 instruction。
- AI assistant 每轮以 Solution: 开头, 并以 Next request. 结束。
- 最后通过 end-of-task token 语义结束。
- 没有 failure signal。

失败检测片段:

```
from camel_role_play import Turn, detect_failures
```

```
bad_turns = [
    Turn("Please implement the plan.", "None", "Instruction: ask me what to do."),
    Turn("Continue.", "None", "I will do that later."),
    Turn("Finish.", "None", "Thank you. You are welcome. Goodbye."),
]

print(detect_failures(bad_turns))
```

你应该看到:

- role_flipping.
- flake_reply.
- infinite_loop.

19. 30 到 60 分钟本地实验

实验目标: 让你亲手感受到 CAMEL 的协议为什么必要。

步骤 1: 跑测试。

```
python learning\multi-agent-orchestration\environment\verify_env.py
python learning\multi-agent-orchestration\src\tests\test_multi_agent.py
```

步骤 2: 单独跑 CAMEL toy。

```
python learning\multi-agent-orchestration\src\camel_role_play.py
```

步骤 3: 打开 camel_role_play.py, 做三个改动中的一个。

改动 A:

- 在 assistant_solution 中去掉 Solution:。
- 观察 failure detector 是否需要增加 assistant repeat 规则。

改动 B:

- 把 max_messages 改成 2。
- 观察 termination_reason 是否变成 max_messages。

改动 C:

- 在 assistant_solution 里让 assistant 返回 Instruction: 开头。
- 观察是否触发 role_flipping。

步骤 4: 对照论文写 5 句话。

模板:

我修改了 role-playing 协议中的某个约束。

这个约束在论文中对应 Figure 2 的某条 prompt rule。

修改后, 对话出现了某类失败。

这说明 CAMEL 的 prompt 不是装饰, 而是为了稳定角色和终止条件。

但这个 toy 只验证机制直觉, 不等于证明真实 LLM 上的效果。

20. 证据链怎么读

证据一: 框架可运行。

- Figure 1 展示 role-playing pipeline。
- 完整交易机器人样例说明 AI user 能持续拆任务，assistant 能持续给 solution。
- 这证明 CAMEL 可以生成多轮协作轨迹。

证据二: 失败模式被识别。

- role flipping。
- repeated instruction。
- flake replies。
- infinite loop。
- termination rules。

这证明作者不是只做 demo，而是在分析 autonomous cooperation 的不稳定性。

证据三: solution quality 评估。

- Table 1 显示 human/GPT4 evaluation 更偏好 CAMEL agent solution。
- 对比对象是 gpt-3.5-turbo single-shot solution。
- 说明多轮协作在这些任务上能产出更完整答案。

证据四: synthetic data 可训练。

- Table 2 显示逐步加入不同领域数据后，对应领域能力提高。
- Table 3 显示 CAMEL-7B 在 HumanEval 上优于 LLaMA-7B 和 Vicuna-7B。

证据五: 附录扩展。

- termination reason distribution。
- prompt ablation。
- critic-in-the-loop。
- embodied agents。
- broader impacts and limitations。

这说明论文试图把 CAMEL 从单个样例扩展为一个研究平台。

21. 局限性

第一，evaluation 可能偏向长答案。

CAMEL 多轮对话总结出的 solution 往往更长、更结构化。human 和 GPT4 judge 可能偏好这种形式，但更长不一定更正确。

第二，对比 baseline 不够完整。

Table 1 主要比较 CAMEL multi-turn solution 和 gpt-3.5-turbo single-shot。更严格的 baseline 应包括:

- single-agent multi-turn with self-planning。
- human-written prompt decomposition。
- ReAct-style planning。
- role-playing without task specifier。
- role-playing without termination rules。
- cost-controlled comparison。

第三，真实 task completion 没有完全验证。

很多任务是方案生成或代码生成。方案看起来好，不等于真实执行成功。

第四，数据质量难评估。

25,000 conversations 很大，但每条是否 factual、safe、useful，需要大量领域专家。

第五，安全风险明显。

Misalignment examples 表明，如果给 autonomous agent system 一个恶意目标，它可能协作生成有害计划。多 agent 不会自动更安全，可能更会分工。

第六，成本和 token 增长很重要。

论文承认 max message limit 和 token limit 是必要保护。大规模生成数据时，成本是核心约束。

22. 对现代 multi-agent 框架的意义

CAMEL 的很多思想后来变成 multi-agent 框架常见组件。

Role assignment:

- AutoGen 里的 assistant/user proxy。
- CrewAI 的角色、目标、backstory。
- MetaGPT 的 PM、Architect、Engineer、QA。
- 本仓库 capstone_coding_crew.py 的 PM/Engineer/Reviewer。

Message protocol:

- 每个 agent 读 history。
- 每个 agent 产出 message。
- message bus 或 graph 控制流转。

Critic or judge:

- reviewer agent。
- supervisor。
- evaluator。
- human approval。

Termination:

- done token。
- max rounds。
- token budget。
- no-progress detector。
- role violation detector。

但 CAMEL 也提醒我们: multi-agent 的漂亮对话不等于可靠工程。后来的框架必须补上:

- state machine。
- tool permission。
- structured output。
- cost accounting。
- observability。
- reproducible evaluation。
- safety policies。

23. 和第 25 篇的连接

AI Agents That Matter 会对 CAMEL 提出一些严厉问题。

成本问题:

- CAMEL 多轮对话比 single-shot 更贵。
- 需要报告 tokens、rounds、API calls。
- 需要比较 accuracy-cost Pareto。

baseline 问题:

- single-shot baseline 可能太弱。
- 应该加 multi-turn single-agent baseline。
- 应该加 task decomposition prompt baseline。

benchmark 问题:

- 方案质量和真实完成率不同。
- GPT4 judge 可能有 verbosity bias。
- Human evaluation 只覆盖有限任务。

复现问题:

- prompt 模板、模型版本、temperature、termination rules 都会影响结果。
- agent conversation 轨迹有随机性。

这是否否定 CAMEL，而是帮你更成熟地使用 CAMEL: 它是基础性探索，不是最终评测标准。

24. 用 AI agent 正确学习这篇

你可以让学习 agent 按下面方式考你。

我正在学习 CAMEL 论文。

请不要只总结多 agent 很有用。

请按以下顺序考我:

1. CAMEL 要替代人类在对话中的哪种工作?
2. Figure 1 中 task specifier, AI user, AI assistant 分别做什么?
3. Inception prompting 的三个 prompt 是什么?
4. 为什么 assistant 必须以 Solution 开头并以 Next request 结尾?
5. CAMEL_TASK_DONE 解决什么问题?
6. role flipping, flake reply, infinite loop 分别是什么?
7. Table 1 证明了什么, 没有证明什么?
8. 为什么多 agent 成本会随轮数快速增长?
9. CAMEL 和 AutoGen/CrewAI/MetaGPT 的关系是什么?
10. 如何在本仓库 camel_role_play.py 中复现一个失败模式?

每次只问一个问题。

我回答后, 请要求我把答案映射到论文图、表或本地代码。

最后让我闭卷画出 Figure 1 的系统图。

25. 闭卷掌握检查

1. CAMEL 解决的不是哪个模型训练问题, 而是交互问题?

2. 为什么 task specifier 对非专家用户重要?
3. AI user 和 AI assistant 的职责边界是什么?
4. 论文公式中的 M_t 、 I_t 、 S_t 分别是什么?
5. Inception prompting 为什么只发生在对话开始前?
6. Figure 2 的哪条规则防止 role flipping?
7. 为什么 Next request. 可以稳定对话控制权?
8. 为什么 CAMEL_TASK_DONE 是必要的?
9. 四类失败模式分别是什么?
10. Table 1 的三组结果如何支持 CAMEL?
11. Table 1 没有排除哪些替代解释?
12. Table 2 的 knowledge emergence 应该谨慎怎么读?
13. HumanEval 上 CAMEL-7B 的结果说明了什么, 没有说明什么?
14. Critic-in-the-loop 和 tree-search-like 决策有什么关系?
15. 为什么多 agent 对话的成本可能接近二次增长?
16. camel_role_play.py 中哪个函数对应 task specifier?
17. detect_failures 对应论文哪一段观察?
18. 你会怎样给 CAMEL 加一个 cost-controlled baseline?
19. CAMEL 对现代 agent 框架最大的遗产是什么?
20. CAMEL 对安全最重要的警告是什么?

26. 最小复述模板

闭卷时可以这样讲:

CAMEL 研究的是怎样让多个 communicative LLM agents 自动协作。它把人类粗略 idea 交给 task specifier 具体化, 再给 AI user 和 AI assistant 分配角色。AI user 负责逐步发 instruction, AI assistant 负责给 Solution, 并用 Next request 把控制权交回。整个对话历史 M_t 不断增长, 直到 CAMEL_TASK_DONE、token limit 或 max message 等条件终止。论文通过 AI Society 和 Code 数据展示 role-playing 能生成多轮协作数据, 并在 human/GPT4 evaluation 中优于 single-shot baseline, 也用生成数据微调 LLaMA 7B 观察能力提升。但 CAMEL 也暴露 role flipping、重复 instruction、flake reply、infinite loop、成本增长和安全风险。它的现代意义是奠定了 role-based multi-agent orchestration 的早期框架, 同时提醒我们多 agent 必须有协议、终止、评测、成本和安全边界。

能讲出这段, 再能跑通 camel_role_play.py 并解释一个失败模式, 这篇就算真正学进去了。